

Architektury systemów komputerowych

Lista zadań nr 10

Na zajęcia 13, 14 i 19 maja

Zadania z tej listy należy rozwiązywać na komputerze z systemem operacyjnym *Linux* dla platformy *x86-64*. Prowadzący zakłada, że zainstalowana dystrybucja będzie bazowała na *Debianie 11*. Do poniższej listy załączono na stronie przedmiotu pliki źródłowe wraz z plikiem *Makefile*.

UWAGA! W trakcie prezentacji rozwiązań należy zdefiniować i wyjaśnić pojęcia, które zostały oznaczone **wytluszczoną** czcionką.

Zadanie 1. Poniżej podano zawartość pliku `swap.c`. Wskaż w nim wszystkie wystąpienia definicji i referencji do **symboli** [1, §7.5]. Dla każdego symbolu wskaż jego zasięg widoczności (tj. lokalny, globalny, zewnętrzny) oraz nazwę **sekcji**, w której go umieszczono (tj. `.text`, `.data`, `.rodata`, `.bss`). Wydając polecenie `make swap.o` wygeneruj **plik relokowalny** i zweryfikuj swoje odpowiedzi na podstawie wydruku z polecenia `nm`, `objdump` lub `readelf`. Do czego *konsolidator* wykorzystuje tablicę symboli?

```
1 extern int printf(          7 static void incr() {          16 void swap(int i) {
2   const char *, ...);      8   static int count = 0;      17   incr();
3 extern long buf[];         9   count++;                  18   long temp = *bufp0;
4                             10 }                            19   *bufp0 = buf[i];
5 long *bufp0 = &buf[0];    11                             20   buf[i] = temp;
6 static double sum = 0.0;   12 void addf(void) {          21 }
                             13   sum += 3.14;
                             14   printf("sum = %f\n", sum);
                             15 }
```

Zadanie 2. Co przechowują sekcje `.strtab` i `.shstrtab` [2, 4-17]? Podejrzij ich zawartość w pliku `swap.o` za pomocą polecenia `readelf` z odpowiednimi opcjami. Opisz znaczenie pól rekordu *symbolu* `Elf64_Sym` [2, 4-18] przechowywanego w **tablicy symboli** [2, 4]. Następnie omów wydruk polecenia `readelf` z opcją `-s` na pliku `swap.o`. Jaka jest pozycja danego symbolu względem początku sekcji?

Zadanie 3. Rozważmy program składający się z dwóch plików źródłowych:

```
1 /* mismatch-a.c */        1 /* mismatch-b.c */
2 void p2(void);             2 #include <stdio.h>
3                             3
4 int main(void) {           4 char main;
5   p2();                   5
6   return 0;               6 void p2(void) {
7 }                          7   printf("0x%x\n", main);
                             8 }
```

Po uruchomieniu program drukuje pewien ciąg znaków i kończy działanie bez zgłoszenia błędu. Czemu tak się dzieje? Skąd pochodzi wydrukowana wartość? Czym różni się **symbol silny** od **słabego**? Zauważ, że zmienna `main` w pliku `mismatch-b.c` jest niezainicjowana. Co by się stało, gdybyśmy w funkcji `p2` przypisali wartość pod zmienną `main`? Co by się zmieniło gdybyśmy w pliku `mismatch-b.c` zainicjowali zmienną `main` w miejscu jej definicji? Czemu dobrym pomysłem jest przekazywanie opcji `-fno-common` do kompilatora?

Zadanie 4. Rozważmy poniższy program składający się z dwóch *jednostek translacji*. Po uruchomieniu kończy się on z błędem dostępu do pamięci. Przy pomocy debuggера `gdb` zatrzymaj się w miejscu wystąpienia awarii i wyjaśnij jej przyczynę. Gdzie została umieszczona stała znakowa `Hello, world!`? Popraw program tak, by zakończył się poprawnie. Nie wolno modyfikować sygnatury procedury `somestr` i pliku `str-a.c`, ani korzystać z dodatkowych procedur. Gdzie umieszczono ciąg znaków po poprawce?

```

1 /* str-a.c */           5 int main(void) {           1 /* str-b.c */
2 #include <stdio.h>       6 char *s = somestr();       2 char *somestr(void) {
3                           7 s[5] = '\0';              3 return "Hello, world!";
4 char *somestr(void);    8 puts(s);                  4 }
                           9 return 0;
                           10 }

```

Zadanie 5. Wydrukuj tablice **rekordów relokacji** z sekcji `.rel.text` i `.rel.data` pliku `swap.o` przy pomocy polecenia `readelf -r`. Na podstawie [1, §7.7.1] wytłumacz uczestnikom zajęć składowe rekordów relokacji `Elf64_Rela` [2, 4-23]. Wyjaśnij jak na podstawie *tablicy rekordów relokacji* polecenie `objdump -d -r swap.o` identyfikuje w zdeasemblowanym kodzie miejsca, które konsolidator będzie musiał uzupełnić w trakcie generowania pliku wykonywalnego. Czy możliwe jest by asembler utworzył sekcję `.rel.bss`?

Zadanie 6. Na podstawie [1, §7.7.2] zreferuj **proces relokowania** referencji do symboli, dla których asembler wygenerował wpisy relokacji typu `R_X86_64_64` i `R_X86_64_32S`. W trakcie tłumaczenia poniższego kodu na asembler kompilator umieścił tablicę skoków dla instrukcji wyboru `switch` w sekcji `.rodata`. W wyniku konsolidacji pliku wykonywalnego zawierającego procedurę `relo3`, została ona umieszczona pod adresem `0x1000`, a tablica skoków pod `0x2000`.

```

1 int relo3(int val) {                                0000000000000000 <relo3>:
2     switch (val) {                                  0: 8d 47 9c          lea    -0x64(%rdi),%eax
3         case 100:                                  3: 83 f8 07          cmp    $0x7,%eax
4             return val + 1;                        6: 77 19              ja     21 <relo3+0x21>
5         case 101:                                  8: 89 c0              mov    %eax,%eax
6         case 103 ... 104:                          a: ff 24 c5 00 00 00 jmpq   *0x0(,%rax,8)
7             return val + 3;                       11: 8d 47 01          lea    0x1(%rdi),%eax
8         case 105:                                  14: c3                retq
9             return val + 5;                       15: 8d 47 03          lea    0x3(%rdi),%eax
10        case 107:                                  18: c3                retq
11            return val + 7;                       19: 8d 47 05          lea    0x5(%rdi),%eax
12        default:                                  1c: c3                retq
13            return val + 11;                      1d: 8d 47 07          lea    0x7(%rdi),%eax
14    }                                              20: c3                retq
15 }                                              21: 8d 47 0b          lea    0xb(%rdi),%eax
                                              24: c3                retq

```

Oblicz wartości, które należy wstawić w miejsca referencji, do których odnoszą się poniższe rekordy relokacji otrzymane poleceniem `objdump -r`.

```

1 RELOCATION RECORDS FOR [.text]:
2 OFFSET          TYPE          VALUE
3 000000000000000d R_X86_64_32S      .rodata
4
5
6 RELOCATION RECORDS FOR [.rodata]:
7 OFFSET          TYPE          VALUE
8 0000000000000000 R_X86_64_64      .text+0x0000000000000011
9 0000000000000008 R_X86_64_64      .text+0x0000000000000015
10 0000000000000010 R_X86_64_64      .text+0x0000000000000021
11 0000000000000018 R_X86_64_64      .text+0x0000000000000015
12 0000000000000020 R_X86_64_64      .text+0x0000000000000015
13 0000000000000028 R_X86_64_64      .text+0x0000000000000019
14 0000000000000030 R_X86_64_64      .text+0x0000000000000021
15 0000000000000038 R_X86_64_64      .text+0x000000000000001d

```

Literatura

- [1] „*Computer Systems: A Programmer's Perspective*”
Randal E. Bryant, David R. O'Hallaron; Pearson; 3rd edition, 2016
- [2] „*System V Application Binary Interface*”
<http://www.sco.com/developers/gabi/latest/contents.html>